



Instituto Tecnológico de Costa Rica

Área Académica de Ingeniería en Computadores

Paradigmas de Programación (CE1106)

Documentación Técnica de tarea de prolog

Profesor:

Marco Rivera Meneses

Estudiantes:

Jestim Espinoza Morales – 2023282666

Primer Semestre 2026

1.1 Descripcion de las reglas implementadas

Tecnica de listas diferenciadas

La gramatica implementada en Logic.pl utiliza listas diferenciadas. En este estilo cada predicado gramatical recibe dos argumentos de lista: S0, la lista de tokens de entrada, y S, la sublista que queda sin consumir. La relacion entre ambas es: S0 = TokensReconocidos ++ S.

Este estilo contrasta con las clausulas DCG (-->) que SWI-Prolog transforma automaticamente en predicados con dos argumentos ocultos. Las listas diferenciadas los explicitan, lo que permite inspeccionar y depurar cada regla directamente. La ventaja practica es que mejor_parseo/3 necesita medir cuantos tokens consume cada intento, calculo que requiere acceso explicito a S0 y S.

Reglas de oracion

El predicado oracion/2 tiene tres clausulas:

```
oracion(S0, S) :- sintagma_nominal(S0, S1), sintagma_verbal(S1, S).
oracion(S0, S) :- sintagma_verbal(S0, S).
oracion(S0, S) :- sintagma_adverbial(S0, S).
```

Primera clausula: oraciones completas con sujeto explicito -- [yo, amo, las, matematicas]. Segunda clausula: oraciones con sujeto elidido -- [amo, las, matematicas]. Tercera clausula: respuestas puramente adverbiales de confirmacion o negacion -- [si], [no, nunca].

Reglas de sintagma_nominal

Cuatro clausulas ordenadas de mayor a menor longitud:

```
sintagma_nominal([P|S], S)          :- pronombre(P).
sintagma_nominal([D, A, N | S], S)  :- determinante(D), adjetivo(A),
es_nombre(N).
sintagma_nominal([D, N | S], S)     :- determinante(D), es_nombre(N).
sintagma_nominal([N|S], S)         :- es_nombre(N).
```

Clausula 1: pronombre personal como nucleo: [yo], [tu]. Clausula 2: det-adj-nombre: [las, hermosas, matematicas]. Clausula 3: det-nombre (la mas comun): [las, matematicas]. Clausula 4: nombre simple: [matematicas]. El orden garantiza que se intente primero el reconocimiento mas largo.

Patrones de sintagma_verbal

12 clausulas ordenadas de mayor a menor longitud. Este ordenamiento es critico: Prolog prueba clausulas en orden y el primer exito es el de mayor cobertura. Si los patrones cortos estuvieran primero, se perderia informacion del resto de la oracion.

```

% Patron 1: V Adv Prep V SN -- "disfruto mucho de resolver problemas"
sintagma_verbal([V1,A,P,V2|S0],S) :-
verbo(V1),adverbio(A),preposicion(P),verbo(V2),sintagma_nominal(S0,S).
% Patron 2: V Prep V SN      -- "disfruto de resolver problemas"
sintagma_verbal([V1,P,V2|S0],S) :-
verbo(V1),preposicion(P),verbo(V2),sintagma_nominal(S0,S).
% Patron 3: V Adv Prep SN    -- "me intereso mucho por las personas"
sintagma_verbal([V,A,P|S0],S) :-
verbo(V),adverbio(A),preposicion(P),sintagma_nominal(S0,S).
% Patron 4: V Prep SN        -- "intereso por las personas"
sintagma_verbal([V,P|S0],S) :-
verbo(V),preposicion(P),sintagma_nominal(S0,S).
% Patron 5: V Adv V          -- "disfruto mucho resolver"
sintagma_verbal([V1,A,V2|S],S) :- verbo(V1),adverbio(A),verbo(V2).
% Patron 6: V V              -- "prefiero escuchar"
sintagma_verbal([V1,V2|S],S) :- verbo(V1),verbo(V2).
% Patron 7: Adv V SN         -- "si amo las matematicas"
sintagma_verbal([A,V|S0],S) :-
adverbio(A),verbo(V),sintagma_nominal(S0,S).
% Patron 8: V Adv SN         -- "amo mucho las matematicas"
sintagma_verbal([V,A|S0],S) :-
verbo(V),adverbio(A),sintagma_nominal(S0,S).
% Patron 9: V SN             -- "amo las matematicas"
sintagma_verbal([V|S0],S) :- verbo(V),sintagma_nominal(S0,S).
% Patron 10: Adv V           -- "si adoro"
sintagma_verbal([A,V|S],S) :- adverbio(A),verbo(V).
% Patron 11: V Adv           -- "amo mucho"
sintagma_verbal([V,A|S],S) :- verbo(V),adverbio(A).
% Patron 12: V solo          -- "amo"
sintagma_verbal([V|S],S) :- verbo(V).

```

Reglas de sintagma_adverbial

```

sintagma_adverbial([A1, A2 | S], S) :- adverbio(A1), adverbio(A2).
sintagma_adverbial([A | S], S)      :- adverbio(A).

```

Primera clausula: pares de adverbios -- "si mucho", "no nunca". Segunda clausula: adverbio solo -- "si", "no". El orden garantiza que el par se intente antes que el singular.

Predicado mejor_parseo/3

Resuelve el problema del ruido lexical: cuando el usuario escribe "hola yo amo las matematicas", el token "hola" no pertenece al lexico reconocido. mejor_parseo/3 intenta parsear desde todos los sufijos posibles y selecciona el de mayor cobertura:

```

mejor_parseo(Tokens, Estructura, MaxCons) :-
    findall(Cons-(S0,Resto),
        ( sufijos(Tokens, S0), oracion(S0, Resto),
          longitud_consumida(S0, Resto, Cons), Cons > 0 ),
        Lista),
    Lista \= [],

```

```
sort(0, @>=, Lista, [MaxCons-(SInicial,RestoFinal)|_]),
Estructura = parse(SInicial, RestoFinal).
```

Fase 1: sufijos/2 genera todos los sufijos de Tokens por backtracking. Fase 2: oracion/2 intenta parsear cada sufijo S0. Fase 3: longitud_consumida/3 mide cuantos tokens se consumieron (filtro Cons > 0 descarta parseos vacios). Fase 4: sort/4 con @>= ordena descendientemente por cobertura; la cabeza es el parseo optimo.

Reglas de polaridad e interpretar/4

```
polaridad(Tokens, negativo) :- hay_negativo(Tokens), !.
polaridad(Tokens, afirmativo) :- hay_afirmativo(Tokens), !.
polaridad(_, desconocido).
```

Dominancia negativa: cualquier marcador de rechazo en la oracion hace que la polaridad sea negativa, independientemente de marcadores positivos. Marcadores negativos: no, nunca, jamas, nada, tampoco, poco (adverbio_neg) y odio, detesto, aburre, aborrezco (verbo_disgusto).

interpretar/4 produce dos estados: ok (parseo exitoso, polaridad conocida, no ambiguo) y no_entendido (cualquier otro caso). El caso ambiguo es una respuesta puramente adverbial afirmativa sin tema ("si mucho"), que segun el enunciado debe pedir repeticion.

1.2 Descripcion de las estructuras de datos desarrolladas

Hechos lexicos de BD.pl

La base de datos lexica define el vocabulario reconocible por el parser. Todos los atomos se almacenan sin tildes y en minusculas:

```
pronombre(P).          % aridad 1. Valores: yo, me, tu, usted, nosotros
determinante(D).       % aridad 1. Valores: el, la, los, las, un, una, mi, mis
nombre(Palabra,Tema).  % aridad 2. Ej: nombre(matematicas, matematicas)
verbo_gusto(V).        % aridad 1. amo, adoro, disfruto, encanta, gusta,
                        prefiero
verbo_disgusto(V).     % aridad 1. odio, detesto, aburre, aborrezco
verbo_neutro(V).       % aridad 1. soy, estoy, podria, tengo, pasar, estudiar
verbo_topico(V,Tema).  % aridad 2. verbo_topico(escuchar, escuchar)
adverbio_afirm(A).     % aridad 1. si, claro, mucho, bastante, totalmente
adverbio_neg(A).       % aridad 1. no, nunca, jamas, nada, tampoco, poco
adverbio_neutro(A).    % aridad 1. muy, tan, bien, mal
preposicion(P).        % aridad 1. a, de, por, con, para, en, sobre
adjetivo(A).           % aridad 1. habil, bueno, excelente, capaz, experto
```

Termino pref(Tema, Polaridad)

pref/2 representa una preferencia del usuario. Tema es uno de los 15 atomos de dominio del sistema (matematicas, tecnologia, personas, problemas, arte, naturaleza, negocios, salud, leyes, construccion, idiomas, ensenanza, escuchar, hablar, ayudar). Polaridad es afirmativo o negativo.

Las preferencias se acumulan en una lista Prolog ordinaria. El invariante de no-duplicados es garantizado por agregar_preferencia/4: verifica miembro(pref(T, _), Prefs) antes de agregar, descartando silenciosamente preferencias sobre temas ya registrados.

```
Ejemplo: [pref(matematicas, afirmativo), pref(tecnologia, afirmativo),
          pref(personas, negativo), pref(problemas, afirmativo)]
```

Estructura profesion(NombreVisible, ListaAfinidades, ListaAntagonias)

```
profesion('Ingenieria en Computadores',
          [matematicas, tecnologia, problemas, negocios],
          [personas, arte, naturaleza, hablar]).
```

NombreVisible: cadena impresa al usuario. ListaAfinidades: temas que suman +2 cuando el usuario los valora afirmativamente y -2 cuando los rechaza. ListaAntagonias: temas que suman +2 cuando el usuario los rechaza y -2 cuando los acepta. El sistema define 11 profesiones.

Termino parse(SInicial, Resto)

parse/2 encapsula el resultado de mejor_parseo/3. SInicial es la sublista de Tokens donde comenzo el parseo exitoso. Resto es la sublista no consumida por oracion/2 (normalmente [] cuando la oracion consume todos los tokens disponibles). Este termino es opaco para interpretar/4, que solo necesita que mejor_parseo/3 tenga exito.

Listas TemasPendientes y Preferencias

```
% TemasPendientes: inicializada en iniciar/0, se consume de izquierda a
derecha
TemasPendientes = [matematicas, tecnologia, personas, arte, salud, naturaleza,
                   problemas, negocios, escuchar, hablar, idiomas, ensenanza,
                   leyes, construccion]

% Preferencias: lista de pref/2, crece en cada respuesta valida
Preferencias = [] % estado inicial
Preferencias = [pref(matematicas, afirmativo), ...] % estado tras primera
respuesta
```

El bucle dialogo/2 descompone TemasPendientes con [Tema | Resto] en cada iteracion. La lista se consume de izquierda a derecha, de modo que los temas se preguntan en el orden de inicializacion.

1.3 Descripcion detallada de los algoritmos desarrollados

1.3.1 Algoritmo de tokenizacion (procesar_linea/2)

Transforma texto libre del usuario en una lista de atomos Prolog normalizados mediante seis transformaciones en cascada:

```
Paso 1 -- string_lower:      convierte todo a minusculas
Paso 2 -- string_chars:     descompone el string en lista de caracteres
Paso 3 -- limpiar/2:        sustituye vocales acentuadas y puntuacion
Paso 4 -- string_chars:     reconstruye el string limpio
Paso 5 -- split_string:     divide por espacios y tabulaciones
Paso 6 -- strings_a_atomos: descarta strings vacios, convierte a atomos
```

Ejemplo para "Yo amo las matematicas.":

```
Entrada:                  "Yo amo las matematicas."
Paso 1 (string_lower):    "yo amo las matematicas."
Paso 3 (limpiar, . -> sp): "yo amo las matematicas "
Paso 5 (split_string):    ["yo", "amo", "las", "matematicas", ""]
Paso 6 (strings_a_atomos): [yo, amo, las, matematicas] <- descarta ""
```

El predicado limpiar/2 recorre caracter a caracter aplicando transformar/2: vocales acentuadas (a,e,i,o,u,n) se convierten a su equivalente sin tilde; signos de puntuacion (.,?!:;Â¿Âi) se convierten a espacio. Los caracteres no contemplados pasan sin cambio.

1.3.2 Algoritmo de parseo (mejor_parseo/3)

Fase 1 -- Generacion de sufijos: sufijos/2 genera por backtracking todos los sufijos de la lista. Para [hola,yo,amo,las,matematicas] genera: [hola,yo,amo,las,matematicas], [yo,amo,las,matematicas], [amo,las,matematicas], [las,matematicas], [matematicas], [].

Fase 2 -- Intento de parseo: para cada sufijo S0 se intenta oracion(S0, Resto). [hola,...] falla porque "hola" no es pronombre, verbo ni adverbio. [yo,amo,las,matematicas] tiene exito: sintagma_nominal consume "yo", sintagma_verbal consume el resto.

Fase 3 -- Medicion de cobertura: longitud_consumida(S0, Resto, Cons) calcula Cons = length(S0) - length(Resto). Para S0=[yo,amo,las,matematicas] y Resto=[], Cons=4. Se filtra Cons > 0.

Fase 4 -- Selección del máximo: findall recopila todos los pares Cons-(S0,Resto) exitosos. sort(0, @>=, Lista, Sorted) ordena descendientemente por cobertura. La cabeza contiene el parseo óptimo, empaquetado como parse(SInicial, RestoFinal).

1.3.3 Algoritmo de inferencia de polaridad

Implementa dominancia negativa: si existe algún marcador negativo, la polaridad es negativa independientemente de los marcadores positivos. Si no hay negativos pero hay afirmativos, la polaridad es afirmativa. Si no hay ninguno, desconocido.

```
polaridad(Tokens, negativo) :- hay_negativo(Tokens), !.  
polaridad(Tokens, afirmativo) :- hay_afirmativo(Tokens), !.  
polaridad(_, desconocido).  
  
hay_negativo([W|_]) :- polaridad_palabra(W, negativo), !.  
hay_negativo([_|T]) :- hay_negativo(T).
```

1.3.4 Algoritmo de extracción de tema

```
tema([W|_], T) :- tema_palabra(W, T), !.  
tema([_|R], T) :- tema(R, T).  
tema_palabra(W, T) :- nombre(W, T).  
tema_palabra(W, T) :- verbo_topico(W, T).
```

Recorre la lista de tokens de izquierda a derecha buscando el primero reconocible como tema. El corte garantiza que se retorna el tema del primer token reconocible sin buscar más. Si no se encuentra ninguno, tema_o_ninguno/2 retorna el átomo ninguno.

1.3.5 Algoritmo de puntuación de profesiones

La puntuación aplica la siguiente tabla para cada preferencia respecto a cada profesión:

Polaridad	Tema en Afinidades	Tema en Antagonías	Aporte
afirmativo	Si	No	+2
afirmativo	No	Si	-2
negativo	No	Si	+2
negativo	Si	No	-2
cualquiera	No	No	0

Los predicados que implementan este algoritmo son:

```
aporte(Tema, afirmativo, Afin, _, 2) :- miembro(Tema, Afin), !.  
aporte(Tema, afirmativo, _, Antag, -2) :- miembro(Tema, Antag), !.
```

```

aporte(Tema, negativo, _, Antag, 2) :- miembro(Tema, Antag), !.
aporte(Tema, negativo, Afin, _, -2) :- miembro(Tema, Afin), !.
aporte(_, _, _, _, 0).

puntuar([], _, _, 0).
puntuar([pref(T, P) | R], Afin, Antag, Score) :-
    aporte(T, P, Afin, Antag, A), puntuar(R, Afin, Antag, RestoScore),
    Score is A + RestoScore.

mejor_profesion(Prefs, MejorNombre, MejorScore) :-
    findall(S-N, (profesion(N, Afin, Antag), puntuar(Prefs, Afin, Antag, S)),
    Lista),
    sort(0, @>=, Lista, [MejorScore-MejorNombre | _]).

```

Ejemplo numerico para Ingenieria en Computadores

Afinidades: [matematicas, tecnologia, problemas, negocios]. Antagonias: [personas, arte, naturaleza, hablar].

Preferencias: [pref(matematicas,afirmativo), pref(tecnologia,afirmativo), pref(personas,negativo), pref(problemas,afirmativo)]

```

pref(matematicas, afirmativo) -> matematicas en Afin, +2
pref(tecnologia, afirmativo) -> tecnologia en Afin, +2
pref(personas, negativo) -> personas en Antag,+2
pref(problemas, afirmativo) -> problemas en Afin, +2
Puntaje total Ingenieria en Computadores = 8

```

1.3.6 Algoritmo de terminacion temprana

```

dialogo(Prefs, _) :- length(Prefs, N), N >= 14, !, recomendar(Prefs).
dialogo(Prefs, _) :- profesion_cubierta(Prefs, _), !, recomendar(Prefs).

profesion_cubierta(Prefs, Nombre) :-
    mejor_profesion(Prefs, Nombre, _), profesion(Nombre, Afin, Antag),
    todos_cubiertos(Afin, Prefs), todos_cubiertos(Antag, Prefs).

todos_cubiertos([], _).
todos_cubiertos([T | Resto], Prefs) :- miembro(pref(T, _), Prefs),
    todos_cubiertos(Resto, Prefs).

```

Primera condicion: 14 o mas preferencias acumuladas -- evita preguntas redundantes. Segunda condicion: la profesion con mayor puntaje ya tiene todos sus temas de afinidades y antagonias cubiertos por preferencias registradas -- el sistema tiene suficiente informacion para recomendar y termina el dialogo anticipadamente.

1.3.7 Validacion del algoritmo de puntuacion

Se verificaron cinco casos de prueba con calculo manual detallado:

Caso	Preferencias resumidas	Profesion esperada	Puntaje calculado	Correcto
1	mat+, tec+, per-, prob+	Ingenieria en Computadores	IC=+8, Psic=-4	Si
2	tec-, per+, prob+, esc+	Psicologia	Psic=+8, IC=-2	Si
3	art+, mat-, tec-, nat+	Artes	Artes=+8, Arq=-2, IC=-8	Si
4	idi+ (irrelevante para IC)	Filologia/Idiomas	IC=0, Fil=+2	Si
5	tec-, mat-	No IC (IC=-4)	Mejor candidato=+4	Si

Comandos de verificacion en SWI-Prolog, se debe consultar el archivo BNF.pl:

```
mejor_profesion([pref(matematicas,afirmativo),pref(tecnologia,afirmativo),pref(
personas,negativo),pref(problemas,afirmativo)],N,S),write(N-S),nl.
```

```
mejor_profesion([pref(tecnologia,negativo),pref(personas,afirmativo),pref(prob
lemas,afirmativo),pref(escuchar,afirmativo)],N,S),write(N-S),nl.
```

El algoritmo cumple la regla +2/-2 en todos los casos. La simetria entre polaridad afirmativa y negativa respecto a afinidades/antagonias produce recomendaciones coherentes con los dos ejemplos del enunciado.

1.4 Problemas sin solucion

Al momento de la entrega, el sistema no presenta problemas que afecten su funcionalidad principal. Las limitaciones conocidas del sistema, junto con sus recomendaciones de correccion, se documentan en la seccion 1.6.

1.5 Plan de Actividades realizadas por estudiante

Actividad	Descripcion	Tiempo estimado	Responsable	Fecha
Diseño de BD.pl	Definición del lexico:	4 horas	Jestim	21 abr 2026

	pronombres, verbos, adverbios, nombres, adjetivos y 11 profesiones con afinidades y antagonias			
Diseno de Logic.pl	Implementacio n de la gramatica BNF con listas diferenciadas, inferencia de polaridad, extraccion de tema y algoritmo de puntuacion	8 horas	Jestim	23 abr 2026
Diseno de BNF.pl	Implementacio n del bucle de dialogo, tokenizacion y lectura de lenguaje natural	6 horas	Jestim	25 abr 2026
Pruebas de integracion	Pruebas del sistema completo con los casos del enunciado y casos borde: ruido lexical, respuestas ambiguas,	3 horas	Jestim	26 abr 2026

	temas desconocidos			
Correccion de bugs	Solucion de problemas de encoding UTF-8, respuestas ambiguas y manejo de fin de entrada	4 horas	Jestim	27 abr 2026
Documentacion tecnica	Redaccion de la documentacion tecnica completa (secciones 1.1 a 1.8)	5 horas	Jestim	28 abr 2026
Revision final	Revision del codigo, documentacion y preparacion de la defensa	2 horas	Jestim	28 abr 2026

1.6 Problemas encontrados

Problema 1: Manejo de codificacion UTF-8 en Windows con swipl-win.exe

Descripcion detallada

SWI-Prolog en Windows utiliza por defecto la codificacion del sistema operativo (comunmente CP-1252). Cuando el usuario escribe texto con vocales acentuadas en la consola swipl-win.exe, la cadena recibida por `read_line_to_string/2` puede no ser una secuencia UTF-8 valida. Esto provoca que `string_chars/2` genere atomos distintos a los almacenados en BD.pl. Por ejemplo, el token producido de "matematicas" con tilde podria diferir del atomo matematicas almacenado en nombre(matematicas, matematicas), causando que `tema_o_ninguno/2` devuelva ninguno aunque el usuario menciona el tema.

Intentos de solucion sin exito

El primer intento fue almacenar los atomos en BD.pl con tildes. No funciono porque la codificacion inconsistente del entorno seguia generando desajustes. El segundo intento fue llamar a `set_stream(user_input, encoding(utf8))` en `iniciar/0`, que produce error en `swipl-win.exe` cuando el stream ya esta abierto con otro modo.

Solucion encontrada

Se adoptaron dos medidas complementarias: (1) La directiva `:- set_prolog_flag(encoding, utf8).` al inicio de `BNF.pl` instruye al interprete a tratar el fuente del archivo como UTF-8. (2) El predicado `limpiar/2` con `transformar/2` recorre caracter a caracter y sustituye cada vocal acentuada por su equivalente sin tilde ANTES del parseo. De este modo el lexico completo en `BD.pl` se almacena sin tildes y la normalizacion ocurre en una unica etapa en `procesar_linea/2`, desacoplando el lexico del encoding del entorno.

Recomendacion

Para versiones futuras, agregar `set_stream(user_input, encoding(utf8))` en `iniciar/0` envuelto en `catch/3` que ignore el error si el stream no acepta la asignacion, mejorando la portabilidad entre plataformas.

Conclusion

La estrategia de almacenar el lexico sin tildes es la decision de diseno mas relevante: desacopla el lexico del encoding y hace que el sistema funcione correctamente independientemente de si el interprete procesa los acentos o no.

Problema 2: Respuestas adverbiales afirmativas ambiguas

Descripcion detallada

El enunciado establece que "Si mucho" debe causar "Me puedes repetir, no entendi.", mientras que "No mucho" debe aceptarse. Esta asimetria tiene justificacion semantica: una respuesta negativa breve es inequivoca (el rechazo no requiere especificidad), mientras que una respuesta afirmativa de solo adverbios no indica sobre que tema se responde afirmativamente.

Sin mecanismo especial, `interpretar/4` devolveria `Estado=ok` con `Pol=afirmativo` y `Tema=ninguno` para `[si,mucho]`, que en `procesar/6` se interpretaria como confirmacion del tema preguntado, registrando informacion posiblemente incorrecta.

Intentos de solucion sin exito

Rechazar toda respuesta sin tema: se descarto porque las respuestas negativas cortas ("no", "no mucho") tambien producen `Tema=ninguno` y deben aceptarse. Agregar la comprobacion en `procesar/6`: descartado porque duplicaria logica que pertenece a la capa de interpretacion.

Solucion encontrada

```
caso_ambiguo(Tokens, afirmativo, ninguno) :- es_solo_adverbial(Tokens).
es_solo_adverbial([]).
es_solo_adverbial([W|R]) :- adverbio(W), es_solo_adverbial(R).
```

caso_ambiguo/3 solo es verdadero cuando la polaridad es afirmativo y el tema es ninguno y todos los tokens son adverbios. La primera clausula de interpretar/4 incluye \+ caso_ambiguo como guarda: si la respuesta es ambigua, falla la primera clausula y el control cae a la segunda (no_entendido).

Conclusion

La solucion mantiene la asimetria requerida por el enunciado sin modificar la gramatica ni la logica de polaridad, y preserva la separacion de capas entre Logic.pl y BNF.pl.

Problema 3: Backtracking y rendimiento del parser

Descripcion detallada

sintagma_verbal/2 define 12 clausulas. mejor_parseo/3 usa findall sobre todos los sufijos, explorando en el peor caso $O(n^2 * c)$ combinaciones para una entrada de n tokens. Con entradas largas y muchas palabras desconocidas el interprete puede intentar un gran numero de unificaciones.

Intentos de solucion sin exito

Agregar cortes (!) en sintagma_verbal/2 reduciria el backtracking dentro de cada parseo, pero romperia la completitud del findall: al comprometerse con la primera rama exitosa, podria perderse un parseo de mayor longitud desde otro sufijo. El objetivo de mejor_parseo/3 es precisamente encontrar el maximo entre todos los posibles.

Solucion encontrada

Se mitigo mediante el ordenamiento de clausulas de mayor a menor longitud: el primer parseo exitoso de cada sufijo suele ser el de mayor cobertura para ese sufijo. Para el caso de uso real (respuestas de 3-8 palabras), el numero total de intentos es manejable en milisegundos.

Recomendacion

Para uso con entradas mas largas: implementar chart parsing (Earley o CYK) con complejidad $O(n^3)$, o limitar la longitud maxima de entrada antes de llamar a mejor_parseo/3.

Conclusion

Para el dominio del proyecto el rendimiento es aceptable. La decision de no introducir cortes preserva la correccion semantica del parser a costa de un espacio de busqueda mayor, tolerable en el contexto universitario.

Problema 4: Bucle potencial cuando el usuario responde fuera del tema

Descripcion detallada

Cuando procesar/6 determina que TemaInferido es diferente al Tema preguntado, devuelve NuevosTemas = [Tema | Resto], repitiendo la misma pregunta. Si el usuario responde

sistematicamente con temas distintos al preguntado, el sistema queda en un ciclo sin mecanismo de escape.

Solucion encontrada (parcial)

No se implemento solucion completa. La mitigacion indirecta es que las condiciones de terminacion temprana ($N \geq 14$ preferencias, o $\text{profesion_cubierta}/2$) pueden interrumpir el bucle si se cubren suficientes otros temas. No garantiza terminacion si el tema bloqueante es uno de los primeros en la lista.

Recomendacion

Agregar un argumento Intentos a procesar/6 e incrementarlo cada vez que se repite la pregunta. Al alcanzar un umbral (2-3), saltar al siguiente tema en Resto registrando el tema como sin preferencia o simplemente omitiendolo.

Conclusion

Este problema representa la limitacion funcional mas significativa de la version actual. No se manifiesta en sesiones normales con usuarios coherentes, pero constituye un caso de fallo sin manejar que debe resolverse antes de un despliegue en produccion.

Problema 5: Ausencia de manejo del token [fin] en el bucle de dialogo

Descripción detallada

El predicado leer_oracion/1 detecta la condicion de fin de archivo (end_of_file) y en ese caso unifica Tokens = [fin]. Sin embargo, la ultima clausula de dialogo/2 no contemplaba ese caso: llamaba directamente a procesar/6 con la lista [fin], que a su vez llamaba a interpretar/4. El token fin no pertenece al lexico reconocido (no es pronombre, nombre, verbo, adverbio ni preposicion), por lo que mejor_parseo/3 fallaba, interpretar/4 devolvía Estado = no_entendido, y procesar/6 imprimía "Me puedes repetir, no entendí." y reinsertaba el mismo tema en la lista de pendientes. Al volver a llamar leer_oracion/1, el stream continuaba cerrado y devolvía [fin] otra vez, creando un bucle infinito hasta que el proceso era interrumpido externamente.

Solucion encontrada (recomendada, no implementada)

```
dialogo(Prefs, [Tema | Resto]) :-
    preguntar(Tema), leer_oracion(Tokens),
    ( Tokens = [fin] ->
        recomendar(Prefs)
    );
    procesar(Tokens, Tema, Prefs, Resto, NuevasPrefs, NuevosTemas),
    dialogo(NuevasPrefs, NuevosTemas)
).
```

Cuando se recibe [fin], el sistema llama a recomendar/1 con las preferencias acumuladas hasta ese momento y termina de forma ordenada. Si no se acumuló ninguna preferencia, recomendar/1 ya

maneja ese caso imprimiendo un mensaje informativo. El cambio no modifica la signatura de ningun predicado existente.

Conclusion

La correccion consistió en una sola línea de código Prolog y no requirió modificar la signatura de ningún predicado existente. El problema surgía por una asuncion implícita en el diseño original: que el usuario siempre provee entrada interactiva valida. El mecanismo de deteccion de `end_of_file` en `leer_oracion/1` estaba correcto desde el inicio; la información simplemente no se propagaba a `dialogo/2`. Con la corrección implementada, el sistema termina con una recomendacion coherente tanto en uso interactivo como con entrada redirigida desde archivo o en entornos de prueba automatizada.

Fuentes consultadas para esta seccion: Bratko (2012) Prolog Programming for Artificial Intelligence, cap. 3-4 (backtracking y cortes); Clocksin y Mellish (2003) Programming in Prolog, sec. 5.3 (manejo de streams); SWI-Prolog Reference Manual -- `set_prolog_flag/2`, `read_line_to_string/2`, `findall/3`, `sort/4`.

1.7 Conclusiones y Recomendaciones del proyecto

Conclusiones

El desarrollo de OrientadorCE demostro en la practica que el paradigma logico es una herramienta natural para implementar sistemas expertos. La representacion del conocimiento de dominio como hechos Prolog en `BD.pl` y la separacion de las reglas de inferencia en `Logic.pl` respecto a la interfaz `BNF.pl` produce una arquitectura donde cada capa puede modificarse independientemente: agregar una profesion o ampliar el lexico no requiere tocar el codigo de parseo ni el bucle de dialogo.

La implementacion de la gramatica BNF con listas diferenciadas demostro que las gramaticas libres de contexto son una herramienta poderosa para procesar lenguaje natural acotado con herramientas puramente logicas. La unificacion de Prolog actua directamente como mecanismo de reconocimiento de patrones: cada clausula de sintagma_verbal/2 es literalmente una regla de produccion BNF. El predicado `mejor_parseo/3` extiende el parser para manejar ruido lexical, haciendolo robusto ante entradas imperfectas.

La gestion de listas como estructura de datos central valido el tercer objetivo especifico. La lista de preferencias, la lista de temas pendientes, las listas de afinidades y antagonias, y las listas diferenciadas de la gramatica son todas listas Prolog ordinarias manipuladas con recursion, unificacion y predicados estandar (`miembro/2`, `length/2`, `findall/3`, `sort/4`). No se requirio ninguna estructura de datos externa ni biblioteca adicional.

La regla de dominancia negativa en la inferencia de polaridad resulto una heurística adecuada para el español conversacional: el usuario raramente expresa rechazo de manera ambigua, y cuando aparece un marcador negativo en la oración la intención de rechazo es dominante.

Recomendaciones

1. Ampliar el léxico en `BD.pl` con sinónimos y variantes morfológicas (matemática/matemáticas, tecnológico/tecnología) para mejorar la cobertura del parser sin modificar `Logic.pl`.
2. Implementar un contador de reintentos en `dialogo/2` para evitar el bucle potencialmente infinito con respuestas fuera del tema (ver sección 1.6, Problema 4).
3. Agregar manejo del token `[fin]` en `dialogo/2` para terminar graciosamente cuando la entrada se cierra (ver sección 1.6, Problema 5).
4. Considerar DCG en lugar de listas diferenciadas para simplificar la gramática en futuras versiones, ya que SWI-Prolog tiene soporte nativo con `phrase/2`.
5. La arquitectura de tres capas es extensible a otros dominios cambiando únicamente el contenido de `BD.pl` y la lista de temas en la inicialización de `BNF.pl`.

1.8 Bibliografía consultada en todo el proyecto

1. Monferrer, T. E., Toledo, F., y Pacheco, J. (2001). El lenguaje de programación Prolog. Valencia. [Referencia principal del curso CE3104. Empleada para convenciones de estilo Prolog, listas, corte y el Ejemplo 6.2 sobre gramáticas con listas diferenciadas.]
2. SWI-Prolog Reference Manual (2024). Recuperado de <https://www.swi-prolog.org/pldoc/>
[Predicados consultados: `read_line_to_string/2`, `string_lower/2`, `string_chars/2`, `split_string/4`, `atom_string/2`, `set_prolog_flag/2`, `findall/3`, `sort/4`, `length/2`, `write/1`, `flush_output/0`, `consult/1`]
3. SWI-Prolog DCG Documentation (2024). <https://www.swi-prolog.org/pldoc/man?section=DCG>
[Consultado para comprender la relación entre DCG y listas diferenciadas y fundamentar la decisión de implementar la gramática con listas diferenciadas explícitas.]
4. Bratko, I. (2012). Prolog Programming for Artificial Intelligence (4a ed.). Addison-Wesley. [Fundamentos de sistemas expertos en Prolog, representación del conocimiento con hechos y reglas, técnicas de búsqueda con backtracking.]
5. Sterling, L., y Shapiro, E. (1994). The Art of Prolog (2a ed.). MIT Press.

[Listas diferenciadas, findall/3, acumulacion de resultados y patron de recursion
con acumulador aplicado en puntuar/4.]

6. Russell, S., y Norvig, P. (2020). Artificial Intelligence: A Modern Approach
(4a ed.). Pearson. [Referencia conceptual para sistemas expertos, capitulo 7.]

1.9 Enlace de GitHub:

<https://github.com/JestimEM/TareaProlog-OrientadorCE.git>